Task 3: Building Airflow ETL Pipeline

Course: MLOps (7th Semester) | **Student Name:** Shoaib Asif
Repository: https://github.com/shoaib-asiff/23L0710_task3

Astro CLI Environment & Container Status

Ran astro dev ps to check the status of all local services. Airflow scheduler, webserver, triggerer, postgres, mongodb, and mongo-express are all running on the same docker network.

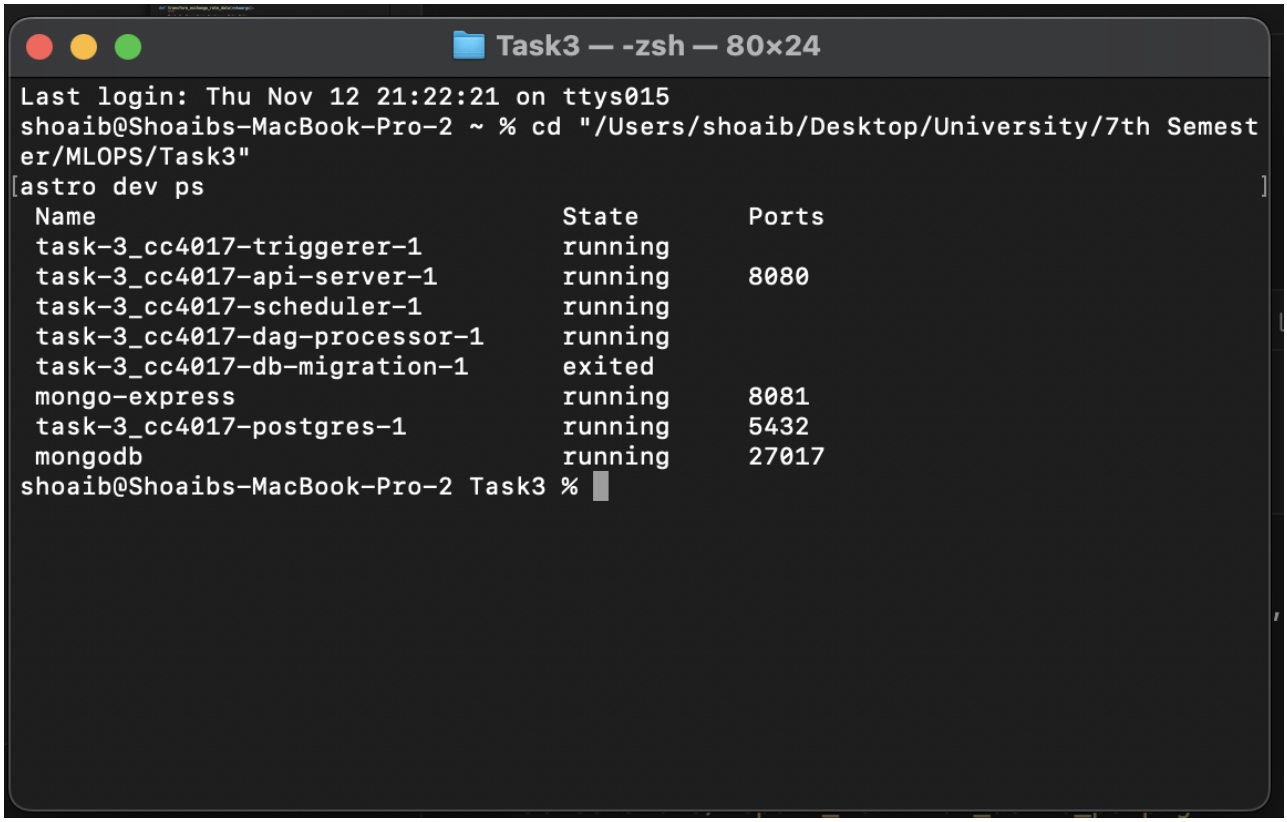


Figure 1: Astro CLI status showing all containers running.

Airflow DAG Execution & Task States (Run 1)

Checked the task states for the pipeline run using Astro CLI. All three tasks (extract_currency_rates, transform_exchange_rate_data, and load_data_into_mongodb) finished with state success.

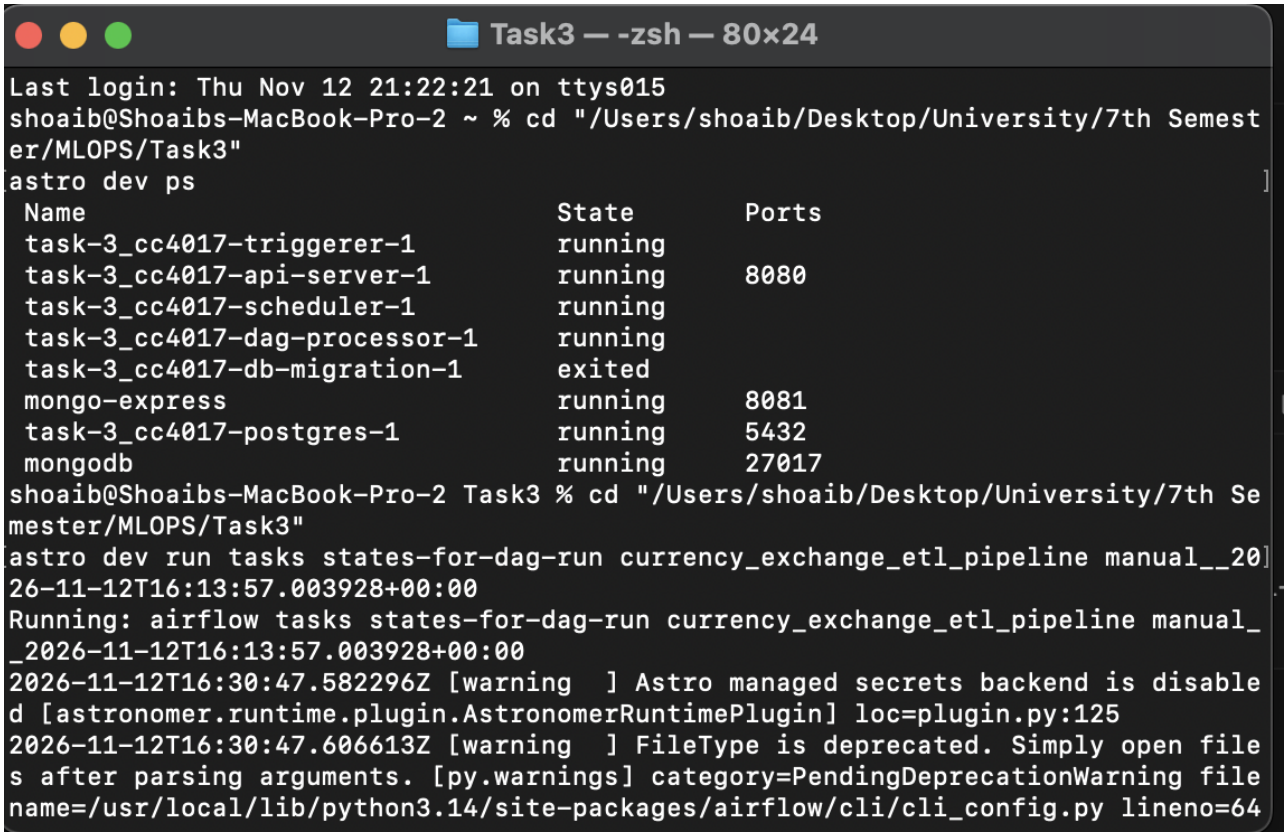


Figure 2: Terminal output showing all three tasks succeeded.

Airflow DAG Repeat Execution (Run 2)

Triggered the pipeline again to verify repeated execution. All tasks completed successfully without errors.

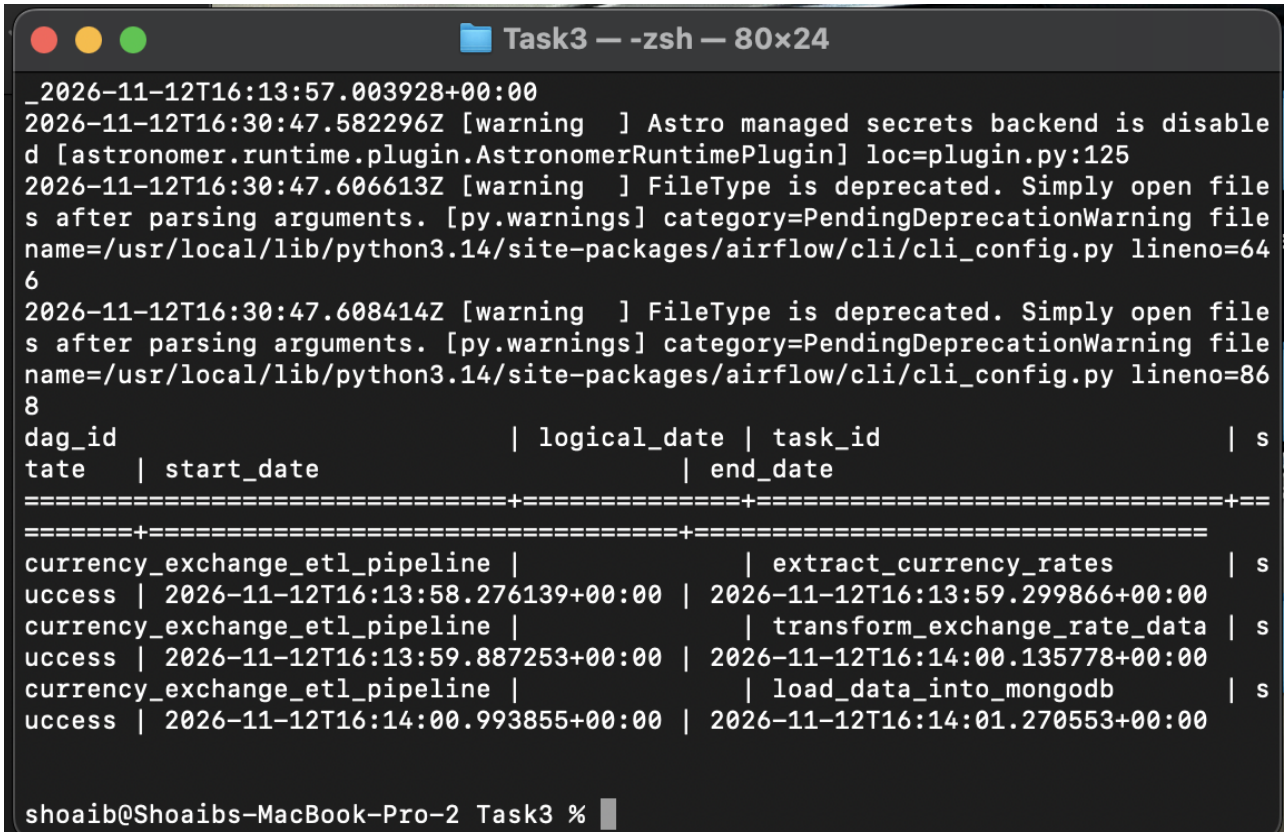


Figure 3: Second DAG run completing with all tasks successful.

Airflow Web UI - Grid View

Opened the Airflow UI at http://localhost:8080. The grid view shows consecutive green boxes confirming successful runs for each task in the pipeline.

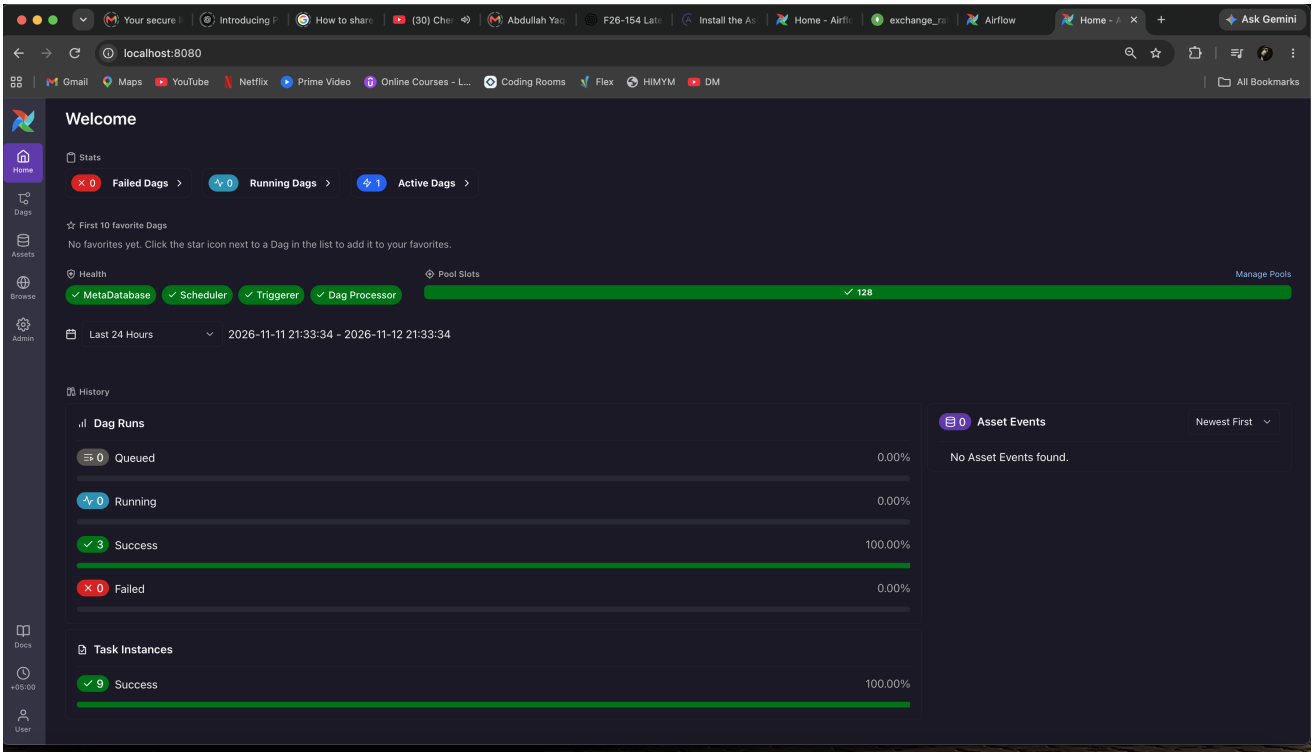


Figure 4: Airflow Grid view showing successful execution history.

Airflow Web UI - Graph View & Task Dependencies

Graph view in Airflow UI showing the task dependencies: extract_currency_rates >> transform_exchange_rate_data >> load_data_into_mongodb.

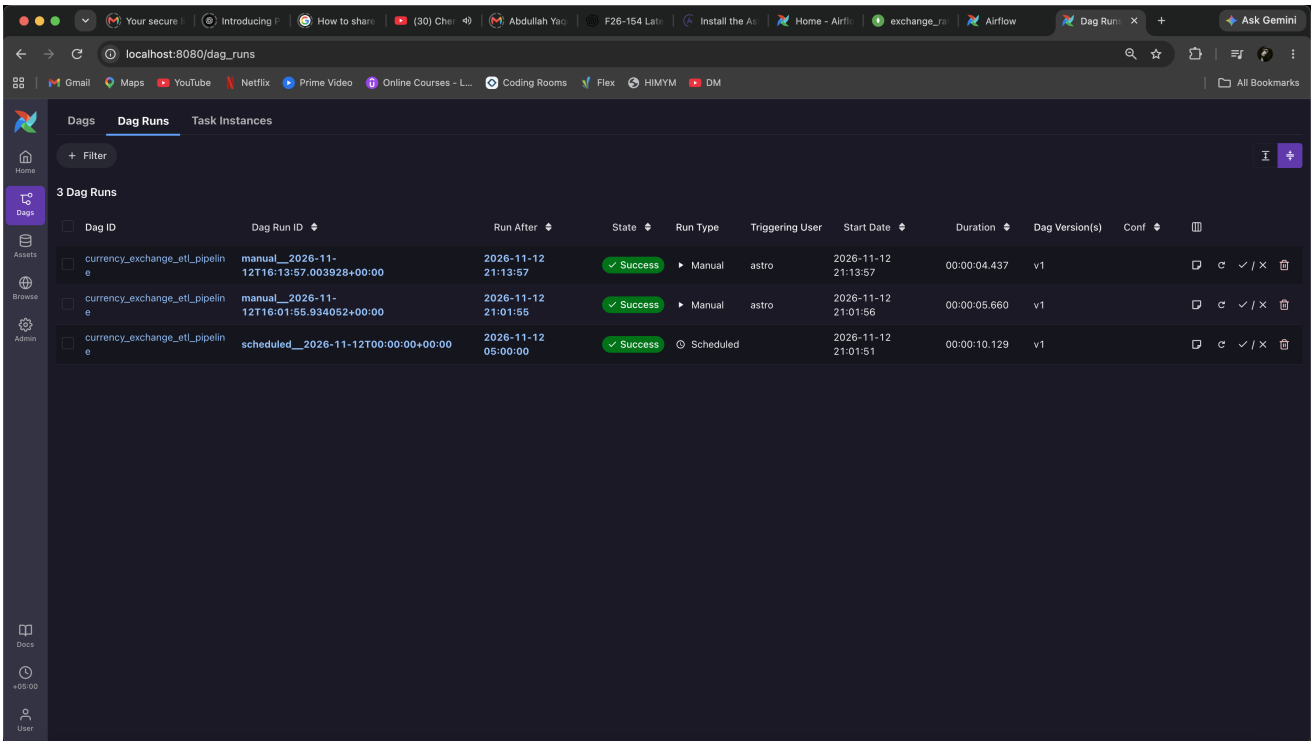


Figure 5: DAG graph view showing the sequential flow of tasks.

Airflow Web UI - DAG Run Details

Detailed view of the DAG run showing execution timestamp, status, and duration.

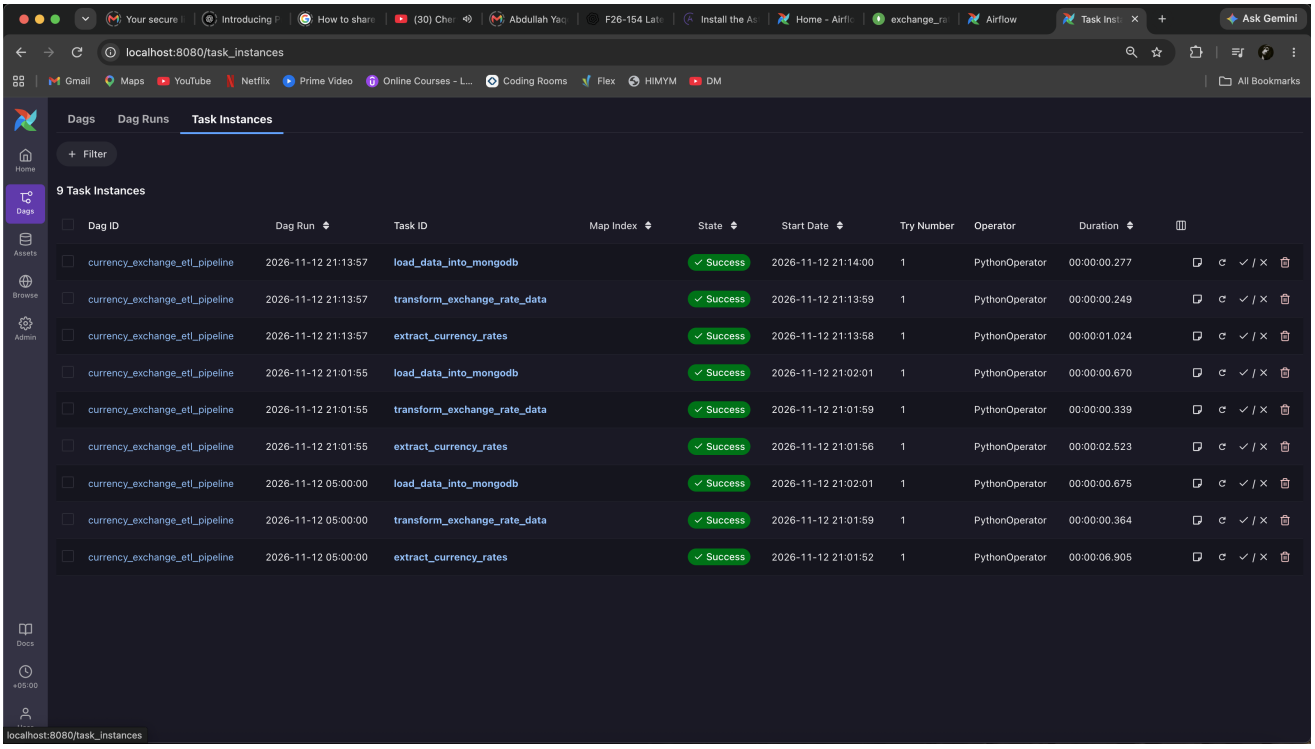


Figure 6: Run details and timing in Airflow UI.

Airflow Named XCom Table

Navigated to Admin > XComs in Airflow to verify data passing between tasks. The raw API data is stored under key 'raw_currency_data' and the transformed records are stored under 'transformed_currency_data'.

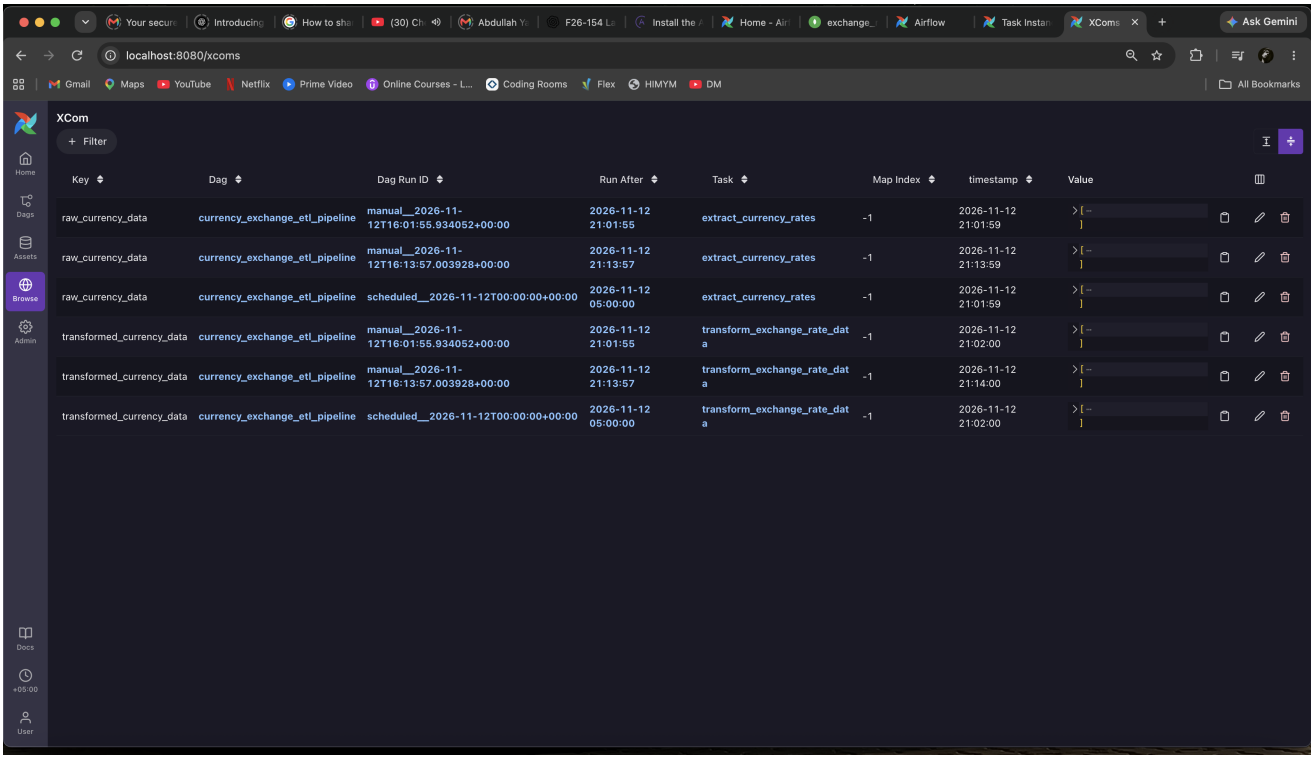


Figure 7: Airflow XCom records showing the custom named keys used.

Mongo Express Web UI - Database Overview

Opened Mongo Express at http://localhost:8081 to inspect MongoDB. The database 'currency_db' and the collection 'exchange_rates' were created successfully.

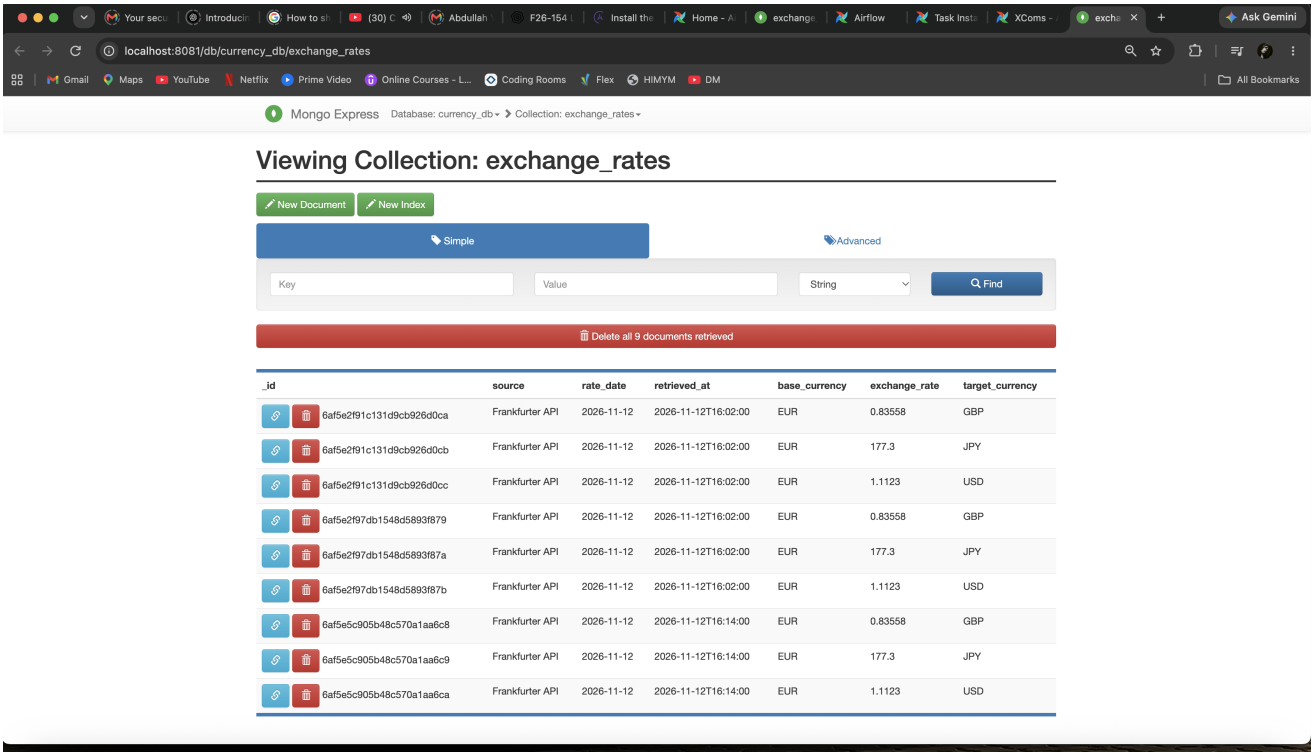


Figure 8: Mongo Express dashboard showing currency_db database.

Mongo Express - Stored Documents View

Viewing the documents loaded into 'exchange_rates'. Each record contains base_currency, target_currency, exchange_rate, rate_date, source, and retrieved_at matching the required format.

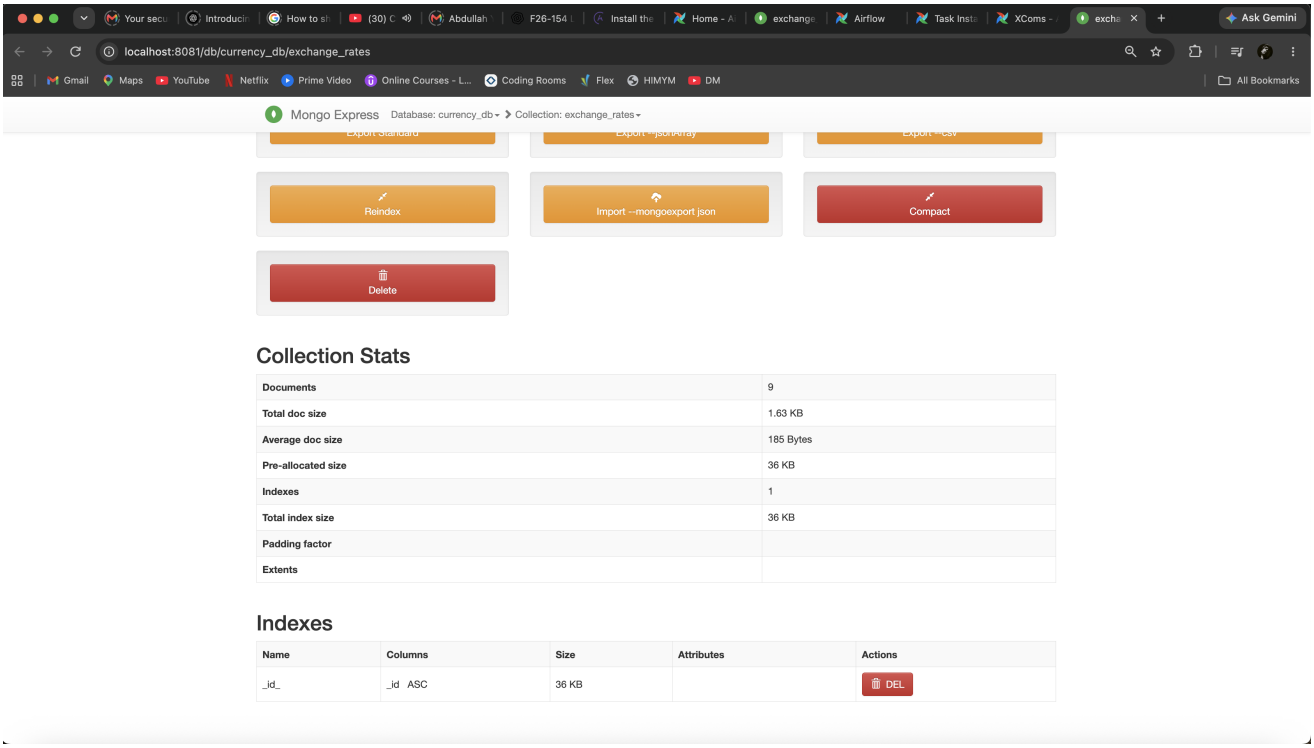


Figure 9: Inserted documents displayed in Mongo Express.

MongoDB Terminal Verification (mongosh)

Connected to the MongoDB container using mongosh and queried the exchange_rates collection to verify the records directly in the database.

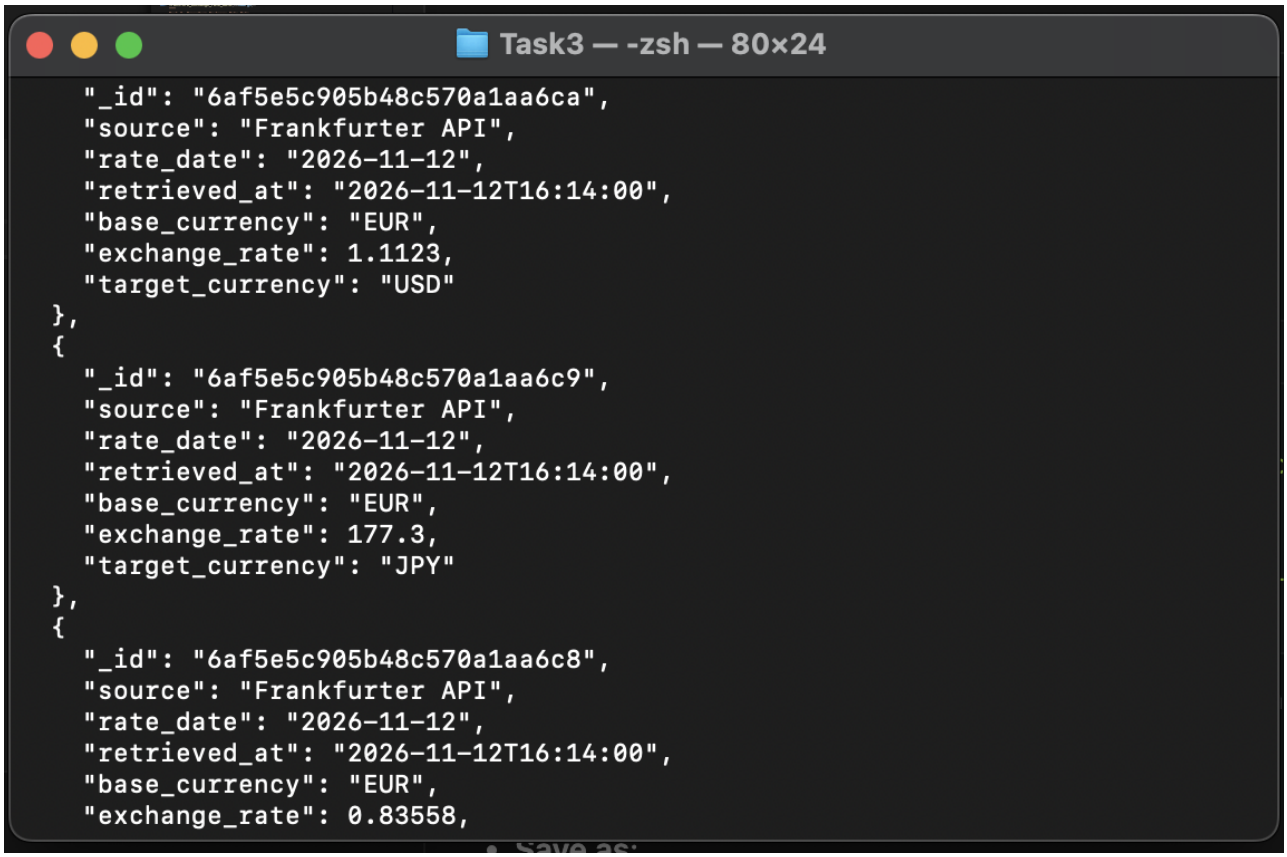


Figure 10: Querying MongoDB from the terminal using mongosh.